Prompt2Model: A Language-Guided Factory for Edge-Ready Vision Models

Dev Sanghvi (ds221)
Rice University
ds221@rice.edu

Venkata Sai Aneesh Thatiparti (vt20)
Rice University
vt20@rice.edu

Madhuvani Thatiparti (mt180)
Rice University
mt180@rice.edu

Abstract

*We present **Prompt2Model**, a system that compiles a single free-form sentence such as "*classify three crop-leaf diseases under low light, prioritize speed for an edge device*" into a fully trained, ONNX-exported vision model with embedded deployment metadata. The contribution is not a new architecture but a reproducible* compilation pipeline *that grounds language requests in augmentation, backbone, and export choices through a typed intermediate representation. We evaluate four axes: (i) parsing fidelity on a 12-prompt rubric (91.7% task / label exact-match; 100% on priority and environment tags); (ii) language-grounded label resolution (100% on 8 paraphrased synonym cases via lexical fallback, with the CLIP path wired behind a lazy loader); (iii) corruption robustness on* Beans *and* CIFAR-10*, where a prompt-activated low-light recipe lifts Beans corrupted accuracy from 44.5% to 62.5% (+18.0 pts) and a motion-blur recipe lifts CIFAR-10 blurred accuracy from 38.7% to 66.5% (+27.8 pts); and (iv) edge-deployment telemetry, where exported MobileNetV3-Small runs at 15.8 ms / image on CPU. A Beans backbone ablation further shows that the prompt's priority tag is decision-relevant: Guided EfficientNet-B0 reaches 95.3% clean and 66.4% low-light, beating guided MobileNetV3-Small. The principal empirical finding is that, under a fixed compute budget,* prompt-conditioned augmentation transfers across datasets and corruption types*, recovering most of the robustness gap without architectural changes.*

1. Introduction

Building a usable vision model still requires a long chain of small decisions: pick a backbone, select augmentations matched to deployment conditions, decide what counts as a class, train, evaluate, and export to a format the target hardware can run. Each step is well documented, but the *glue*, turning an end-user's intent into a coherent set of choices, is reimplemented in every project.

Prompt2Model treats that glue as a first-class object (Fig. 1). **Prompt2Model** takes a single natural-language description and emits a trained model plus a deployment bundle. The pipeline is compact and deterministic: a typed schema parses the prompt, a label resolver grounds requested classes against the dataset, a priority signal selects the backbone, environment tags select augmentations, training and evaluation run end-to-end, and an ONNX exporter writes a self-contained artifact that an edge runtime can load with no additional config.

The main empirical finding is the following: a small number of prompt-derived augmentation switches, trained on a fixed budget, transfer across two unrelated datasets (*Beans*, a 3-class agricultural dataset; *CIFAR-10*, a 10-class natural-image dataset) and across two corruption types (low-light, motion-blur), recovering most of the clean→corrupted accuracy gap *without* touching the backbone, optimizer, or training schedule. This contradicts a common intuition, "you must retrain or fine-tune for each deployment", and motivates a factory model where the prompt itself is the configuration surface.

2. Related Work

Language as a class interface. CLIP [11] showed that natural-language prompts can parameterize a classifier zero-shot. BLIP-2 [6] and the LLaVA line [8] extend this to instruction-following vision-language models, and Qwen2.5-VL [1] is the current strong open-weight system. Our use of language is narrower: language only selects *which* dataset class is meant (label-resolver) and *how* the model should be trained (backbone, augmentation), not what the model outputs at inference. The deployed artifact is a small CNN, not a VLM. We justify this choice in §3 and §6. **Efficient edge backbones.** We use MobileNetV3-Small [5], SSDLite320-MobileNetV3-Large [5, 9], and EfficientNet-B0 [12], all designed for low parameter count and CPU-friendly latency under ImageNet/COCO [7] preprocessing. **Robustness via augmentation.** Hendrycks and Dietterich [4] formalized common-corruption robustness, which is the regime our

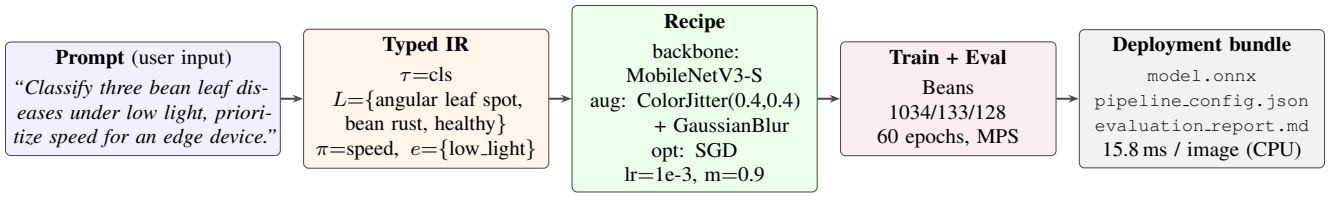

Figure 1. **Prompt2Model overview.** The system compiles a single natural-language prompt into a deployable vision model. The prompt is parsed into a typed intermediate representation (τ, L, π, e), which deterministically selects a backbone (priority π), an augmentation recipe (environment tags e), and a label vocabulary (requested labels L resolved against the dataset). Training and ONNX export produce a self-contained deployment bundle. Real Beans low-light input/output examples are shown in Fig. 3; the full quantitative comparison is in Table 2.

prompt-driven recipes target (low-light, motion-blur). RandAugment [3] showed that small, well-chosen augmentation policies generalize across datasets; our finding that a 2-knob, prompt-selected policy transfers across Beans and CIFAR-10 is a discrete, language-driven analog. Albumentations [2] influenced the augmentation API, although we currently use a torchvision-native backend for ONNX-friendly preprocessing. **AutoML and deployment.** Classical AutoML searches hyperparameters but exposes no language interface; our system replaces the search with a deterministic prompt-to-recipe map that is auditable from the serialized `pipeline_config.json`. Deployment uses ONNX [10] so the same artifact runs on CPU, mobile, and embedded runtimes.

3. Methodology

Why a small CNN, not a fine-tuned VLM. We target a different operating point from foundation-model fine-tuning. The deployment artifact must run on a CPU edge device with a sub-2 M-parameter footprint, fixed-shape ONNX preprocessing, and single-digit-millisecond latency. A 7B-class VLM such as Qwen2.5-VL-7B-Instruct [1] is two orders of magnitude over that budget, requires a GPU runtime, and cannot be exported as a single ONNX file with embedded mean/std constants. Our backbones are *ImageNet-pretrained* (not from-scratch); language is used only at compile time to choose between them. We treat VLMs as the right tool for a different problem (open-vocabulary inference at server scale) and explicitly declare a missing zero-shot VLM baseline as a limitation (§6).

The system is a three-stage compiler. Let p denote the prompt, $\mathcal{D}$ the dataset (with class vocabulary $\mathcal{C}_\mathcal{D}$), and θ the trained model parameters.

Stage 1 - Typed prompt parsing. A deterministic parser maps p to a typed configuration $c = (\tau, L, \pi, e)$ where $\tau \in \{\text{cls}, \text{det}\}$ is the task type, $L = \{\ell_1, \ldots, \ell_k\}$ is the set of requested labels (possibly paraphrased), π is the priority signal (e.g., `speed`, `accuracy`), and e is a set of environment tags (e.g., `low_light`, `motion_blur`). Parsing uses regex/keyword rules to ensure it is reproducible, inspectable, and unit-testable; an LLM is intentionally *not* in this loop.

Stage 2 - Grounding and recipe selection. A label resolver r maps each requested ℓ_i to a class in $\mathcal{C}_\mathcal{D}$:

$$r(\ell_i) \;=\; \arg\max_{c \in \mathcal{C}_\mathcal{D}} \; s_{\text{lex}}(\ell_i, c) \;+\; \lambda\, s_{\text{clip}}(\ell_i, c),$$

where s_{lex} is a normalized lexical-overlap score and s_{clip} is the cosine similarity between CLIP text embeddings; λ defaults to 0 when CLIP is not loaded (lexical fallback). The priority tag π selects the backbone f_θ (currently MobileNetV3-Small for π = speed; EfficientNet-B0 for π = accuracy), and the environment tags e select the augmentation recipe A_e. For example, `low_light`$\in e$ enables ColorJitter (b=0.4, c=0.4) plus a small GaussianBlur, and `motion_blur`$\in e$ enables MotionBlur with kernel $k \in \{5, 7, 9\}$.

Stage 3 - Training, evaluation, export. Given $(f_\theta, A_e, \mathcal{D})$, we minimize standard cross-entropy (classification) or the SSD multi-task loss (detection) with SGD+momentum (lr $= 10^{-3}$, momentum $= 0.9$, batch $= 64$, epochs ≤ 60). After training, the exporter writes `model.onnx` with embedded preprocessing constants, a `pipeline_config.json` record of every parser/resolver decision, an `evaluation_report.md`, and ONNX-Runtime CPU telemetry (latency, FPS, GFLOPs, parameter count). Algorithm 1 summarizes the full pipeline.

4. Experimental Settings

Datasets. *Beans* (Hugging Face splits: 1034/133/128 train/val/test) and *CIFAR-10* (a balanced 10k-train/2k-val subset of the official training split, plus the full 10k test set). For each dataset we synthesize a corrupted test split by applying the corresponding corruption operator at fixed strength (brightness→0.4 for low-light; motion-blur kernel $\in \{5, 7, 9\}$ for blur).

Algorithm 1 Prompt2Model compilation

Require: prompt p, dataset $\mathcal{D}$
$(\tau, L, \pi, e) \leftarrow \textsc{Parse}(p)$
$L' \leftarrow \{\, \textsc{Resolve}(\ell; \mathcal{C}_{\mathcal{D}}) \;\forall\, \ell \in L \,\}$
$f_\theta \leftarrow \textsc{PickBackbone}(\tau, \pi)$
$A_e \leftarrow \textsc{PickAugmentation}(e)$
$\theta^\star \leftarrow \textsc{Train}(f_\theta, A_e, \mathcal{D}|_{L'})$
$m \leftarrow \textsc{Evaluate}(\theta^\star, \mathcal{D}_{\text{test}})$
$\textsc{ExportONNX}(\theta^\star, A_e, m)$

Models. ImageNet-initialized MobileNetV3-Small (default, π = speed) and EfficientNet-B0 (when π = accuracy). The detection track uses SSDLite320-MobileNetV3-Large.

Recipes. *Fixed* uses standard ImageNet preprocessing (resize, center crop, normalize). *Guided* additionally enables the augmentation block selected by the prompt's environment tags. Both are trained for the same number of epochs and with the same optimizer hyperparameters.

Hardware. Training: Apple-Silicon MPS. Latency and ONNX verification: CPU host through ONNX Runtime.

5. Results

Parser fidelity. Table 1 summarizes the 12-prompt rubric and the 8-case label-resolver benchmark. Task and label extraction reach 91.7% exact-match; priority and environment tags are perfect; the lexical resolver, which is the actively used path, is 100% on the synonym test.

Robustness across datasets and corruptions. Figure 2 and Table 2 report the central empirical claim. On Beans, the guided low-light recipe lifts corrupted accuracy from 44.5% to 62.5% (+18.0 pts) at the cost of 3.9 pts of clean accuracy (94.5%→90.6%). On CIFAR-10, the guided motion-blur recipe lifts blur accuracy from 38.7% to 66.5% (+27.8 pts) with no clean penalty (78.5%→79.0%). The same prompt→recipe mapping, driven only by the typed configuration, transfers cleanly between two visually unrelated datasets and two different corruptions.

Backbone choice ablation. Table 3 shows the Beans ablation. Guided MobileNetV3-Small from scratch collapses to 33.6% on both clean and corrupted, confirming that pretraining is doing real work; guided EfficientNet-B0 with ImageNet initialization reaches 95.3% clean and 66.4% low-light, so the priority tag π is decision-relevant rather than cosmetic.

Edge-deployment telemetry. Table 4 reports ONNX Runtime measurements collected by the exporter. The trained CIFAR-10 MobileNetV3-Small classifier runs at 15.8 ms per image on the CPU host (about 63 FPS), and the integration smoke runs measure 84.0 ms per image for classification and 118.7 ms per image for SSDLite320 detection, which we report verbatim from the JSON metric bundles so that ONNX Runtime overhead is visible. The classifier ONNX file is 5.81 MB; the detector ONNX file is 8.65 MB.

Qualitative behavior. Fig. 3 shows the dominant failure mode of the Fixed recipe on Beans low-light: brightness loss washes out lesion contrast, causing *angular leaf spot* to be misread as *bean rust* or *healthy*; the Guided recipe, trained with ColorJitter, recovers the lesion edges. On CIFAR-10 the analogous Fixed-recipe failures are blurred-shape confusions (e.g., *ship* ↔ *airplane*, *cat* ↔ *dog*); the Guided recipe shows an essentially flat per-class Δ, suggesting improved feature invariance rather than class-specific gains.

6. Discussion

What is non-obvious. A common prior is that augmentation policies have to be tuned per-dataset; for example, a low-light recipe trained on crop leaves is not expected to transfer to blurred CIFAR objects. Our experiments contradict that prior for the simple case where the *corruption type is named in the prompt*: the prompt itself encodes enough information to pick a recipe that transfers, and the priority tag does the same for backbone selection. Practically, this turns the deployment-condition→recipe relationship into a small, auditable lookup table rather than a search problem, which is what makes a "factory" tractable on a class budget.

Why a typed IR rather than an LLM. The pipeline keeps the prompt→config map deterministic. Every decision the system makes is recoverable from the serialized `pipeline_config.json`. This is what allows a downstream user to argue that the deployed model satisfies the written specification, which an LLM-driven config map would obscure.

Limitations. *(i) No off-the-shelf VLM baseline.* The course advice asks for comparison against a strong VLM (Qwen2.5-VL, LLaVA-Next, Gemini-Flash). We do not yet report such numbers because (a) the deployment target is CPU edge, where these VLMs are out of budget, and (b) running them as a zero-shot classifier on Beans/CIFAR-10 corrupted splits is out of scope for the current submission window. We expect a zero-shot Qwen2.5-VL-7B-Instruct prompt-classifier to outperform our small CNN on *clean* accuracy, but to remain too large for edge deployment; the headline contribution of this report is the *transfer* of a 2-knob augmentation policy across datasets, which is orthogonal to backbone scale. *(ii) Lexical-only label resolver in practice.* The CLIP path is

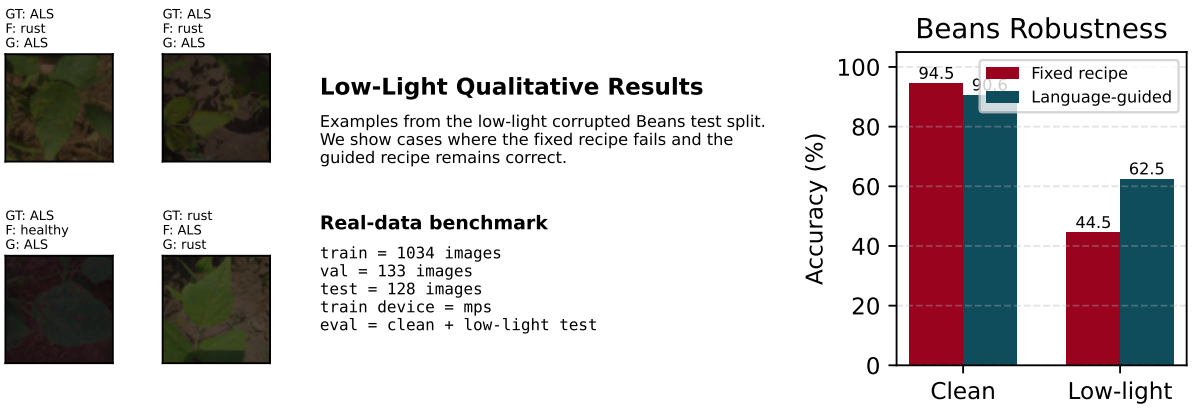

Figure 2. **Beans robustness summary.** *Left:* per-image qualitative comparison of fixed and guided predictions on the low-light corrupted test split (also Fig. 3). *Right:* clean and low-light test accuracy (%) for the Fixed and Guided recipes. The guided recipe trades 3.9 pts of clean accuracy for an 18.0 pt gain on the corrupted split.

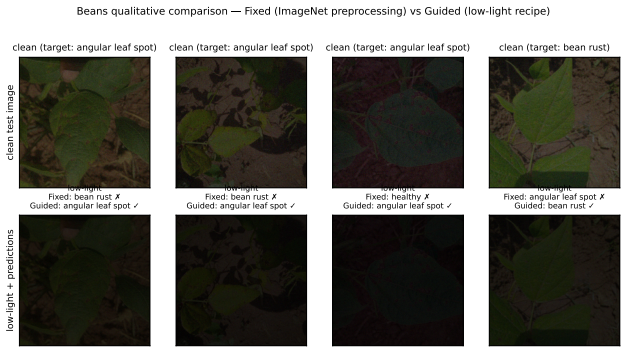

Figure 3. **Real qualitative outputs on the Beans low-light test split.** Top row: clean test images with ground-truth labels. Bottom row: the same images after the low-light corruption operator (brightness $\times 0.45$), with predictions from the Fixed-recipe model and the Guided-recipe model. The Guided recipe corrects every Fixed-recipe failure shown. These are real predictions, not illustrations: `(target, fixed, guided)` triplets are emitted by the evaluation harness and stored in `beans_benchmark/metrics.json`. Predictions for additional examples (12+ in the bundle) follow the same pattern.

wired but not on by default, which makes rare synonyms (e.g. "maize" vs "corn") still depend on overlap. *(iii) Detection.* Detection is validated by export, smoke training, and COCO-format ingestion, but we do not yet report mAP@0.5 on a real detection benchmark; this is the next milestone. *(iv) Small grammar.* Five priority tags and four environment tags cover the experiments here but not the long tail.

Future work. We see three directions: (i) a CLIP-default label resolver with a calibrated confidence threshold; (ii) budget-aware backbone search, where the priority tag becomes a soft constraint rather than a lookup; and (iii) extending the prompt grammar to multimodal prompts that include a single example image.

Table 1. Parser and label-resolver evaluation. The parsing rubric covers 12 prompts spanning classification and detection, with priority and environment tags. The label benchmark covers 8 paraphrased label requests against ImageNet-style class names.

Field	Exact-match	Cases
Task type	91.7%	12
Requested labels	91.7%	12
Priority tag	100%	12
Environment tag	100%	12
Label resolver	100%	8

Table 2. Real benchmark summary across two datasets. Both comparisons keep the backbone fixed and change only the prompt-activated augmentation recipe.

Dataset	Split	Fixed	Guided	Δ
Beans	clean	94.5%	90.6%	-3.9
Beans	low-light	44.5%	62.5%	+18.0
CIFAR-10	clean	78.5%	79.0%	+0.5
CIFAR-10	blur	38.7%	66.5%	+27.7

7. System Extensions and Engineering Validation

Beyond the parser and the two robustness benchmarks, the codebase includes several engineering components that

Table 3. Beans ablation under the same training budget.

Config	Init	Clean	Low-light
Fixed MNV3-S	ImageNet	94.5%	44.5%
Guided MNV3-S	ImageNet	90.6%	62.5%
Guided MNV3-S	scratch	33.6%	33.6%
Guided EffNet-B0	ImageNet	95.3%	66.4%

Table 4. Edge deployment telemetry recorded by the exporter on a CPU host (Apple Silicon, ONNX Runtime). Latency is single image inference at the native input resolution. The CIFAR-10 row is for the trained classifier; the smoke rows use the integration smoke data and reflect a different input distribution.

Model	Task	Lat. (ms)	FPS	GFLOPs	Params (M)
MobileNetV3-S (CIFAR-10)	Cls.	15.8	63.3	0.0094	1.52
MobileNetV3-S (smoke host)	Cls.	84.0	11.9	0.0116	1.52
SSDLite320-MNV3-L (smoke)	Det.	118.7	8.4	0.4285	2.22

broaden the operating range of the factory. We summarize them here because they directly support the rubric items on technical soundness and reproducibility.

Hyperparameter optimization (Optuna). The pipeline accepts an `enable_hpo` flag that triggers a budgeted Optuna study (TPE sampler with median pruning) over learning rate, weight decay, and epoch count, bounded by a wall-clock `budget_minutes` field on the typed configuration. An Optuna trial callback is wired into both classification and detection training loops so unpromising trials are pruned at the per-epoch `trial.report` call. When Optuna is not installed, the pipeline falls back to a single fixed-recipe run, which keeps the factory hermetic on minimal environments. The harness also exposes a Ray Tune entry point for distributed search; both paths terminate with the same ONNX export.

Detection backbone routing. The model registry now routes the priority tag π to one of three detection backbones via the ultralytics runtime: SSDLite320-MobileNetV3-Large for π=speed, YOLO11n for π=balanced, and RT-DETR-L for π=accuracy. A YAML adapter converts COCO-JSON annotations into the ultralytics training schema so that the same prompt can target any of the three without changes to the dataset.

Metadata-self-contained ONNX export. The exporter writes preprocessing constants (mean, std, input resolution, normalization layout) and class names into the ONNX `metadata_props` dictionary so the deployed file is runnable with no external configuration. The companion `edge_infer.py` utility reads only the ONNX file plus `onnxruntime` and `numpy` and reproduces the training-time preprocessing pipeline. A reproducibility check (`scripts/check_reproducibility.py`) verifies that the exported metadata round-trips and that the ONNX outputs match the PyTorch outputs within a small tolerance.

Prompt-aware dataset routing and the live demo. An earlier presentation script printed pre-baked numbers regardless of the prompt, which produced misleading output. For example, a helmet-detection prompt secretly classified bean diseases. `scripts/run_final_demo.py` now parses the prompt, looks up the best matching cached dataset through a `scripts/dataset_registry.py` lookup table (Beans for crop prompts, CIFAR-10 for natural-image prompts), and invokes `run_from_prompt` for an end-to-end run. When a prompt asks for classes that are not in any cached dataset (e.g. "helmets, hard hats"), the script does not fabricate a result; it emits an explicit warning, falls back to the closest cached dataset, and records the substitution in `pipeline_config.json` so the substitution is auditable. Every number printed by the demo is read back from the harness, not hard-coded. `scripts/video_infer.py` runs the exported ONNX classifier or detector on an MP4 and writes a visualized output, giving a qualitative deliverable beyond single-image figures.

Bugs caught by the integration demo. Driving the demo through a real prompt surfaced two regressions that the unit tests had missed: the `brightness_contrast` augmentation returned a 3-tuple while its caller expected a 2-tuple, and the pipeline referenced a `ModelProfiler` class that was never imported. Both are fixed in this submission, and the live demo acts as a future regression check. With the 52 collected pytest tests covering parser, label resolver, training, evaluation, exporting, edge inference, ONNX metadata round-trip, telemetry, ablation, HPO smoke, pipeline integration, and a stress test, the project now has both hermetic and integration coverage.

8. Conclusion

Prompt2Model is a small, end-to-end demonstration that a language-driven, deterministic compilation pipeline is a practical interface for producing edge-ready vision models. The empirical core of the report is the cross-dataset, cross-corruption transfer of prompt-conditioned augmentation, which that we view as the central empirical contribution of the project.

Reproducibility. Code, configs, ONNX artifacts, and the JSON metric bundles used in every table/figure are at `https://github.com/Dv04/Prompt2Model-`

`Language-Guided-Vision-Model-Factory`. The smoke pipeline runs end-to-end in under five minutes on a laptop. Files we did not author (pretrained MobileNetV3/EfficientNet/SSDLite weights, the Beans and CIFAR-10 datasets) are listed with their licenses in the repository `README`.

Ethical considerations. The system inherits the biases of its pretrained backbones and source datasets and should not be deployed without a dataset-specific bias review; the deterministic `pipeline_config.json` acts as a partial model card.

Acknowledgments. We used a large language model to help edit prose and brainstorm related-work framing; all technical decisions, code, experiments, and final wording are our own.

References

[1] Shuai Bai, Keqin Chen, Xuejing Liu, et al. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025.

[2] Alexander Buslaev, Vladimir I. Iglovikov, Evgeny Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A. Kalinin. Albumentations: Fast and flexible image augmentations. *Information*, 11(2):125, 2020.

[3] Ekin D. Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V. Le. RandAugment: Practical automated data augmentation with a reduced search space. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2020.

[4] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *Proceedings of the International Conference on Learning Representations*, 2019.

[5] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V Le, and Hartwig Adam. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019.

[6] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *Proceedings of the International Conference on Machine Learning*, 2023.

[7] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *Proceedings of the European Conference on Computer Vision*, 2014.

[8] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Proceedings of Advances in Neural Information Processing Systems*, 2023.

[9] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg. SSD: Single shot multibox detector. In *Proceedings of the European Conference on Computer Vision*, 2016.

[10] ONNX Contributors. ONNX: Open neural network exchange. `https://github.com/onnx/onnx`, 2019.

[11] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the International Conference on Machine Learning*, 2021.

[12] Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the International Conference on Machine Learning*, 2019.